

# **OAuth 2.0 Authentication**

Penetration Testing Notes

**Eyad Islam El-Taher**

July 8, 2026

# Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>5</b> |
| 1.1      | Why OAuth Matters for Pentesters . . . . .              | 5        |
| <b>2</b> | <b>OAuth Fundamentals</b>                               | <b>5</b> |
| 2.1      | Core Concepts . . . . .                                 | 5        |
| 2.2      | OAuth Components . . . . .                              | 5        |
| 2.3      | OAuth Endpoints . . . . .                               | 6        |
| 2.4      | OAuth Tokens . . . . .                                  | 6        |
| <b>3</b> | <b>OAuth Flows</b>                                      | <b>6</b> |
| 3.1      | Authorization Code Grant . . . . .                      | 6        |
| 3.2      | Implicit Grant . . . . .                                | 7        |
| 3.3      | Other Flows . . . . .                                   | 7        |
| <b>4</b> | <b>Common Misconceptions</b>                            | <b>7</b> |
| <b>5</b> | <b>Attack Surface</b>                                   | <b>7</b> |
| <b>6</b> | <b>OAuth Vulnerabilities</b>                            | <b>7</b> |
| 6.1      | Authentication Bypass via OAuth Implicit Flow . . . . . | 7        |
| 6.1.1    | Theory . . . . .                                        | 7        |
| 6.1.2    | Why It Happens . . . . .                                | 8        |
| 6.1.3    | Attack Prerequisites . . . . .                          | 8        |
| 6.1.4    | Step-by-Step Exploitation . . . . .                     | 8        |
| 6.1.5    | HTTP Request Example . . . . .                          | 8        |
| 6.1.6    | Real-World Scenario . . . . .                           | 8        |
| 6.1.7    | Impact . . . . .                                        | 8        |
| 6.1.8    | Detection Methodology . . . . .                         | 8        |
| 6.1.9    | Important Notes . . . . .                               | 8        |
| 6.2      | Improper <code>redirect_uri</code> Validation . . . . . | 9        |
| 6.2.1    | Theory . . . . .                                        | 9        |
| 6.2.2    | Why It Happens . . . . .                                | 9        |
| 6.2.3    | Attack Prerequisites . . . . .                          | 9        |
| 6.2.4    | Step-by-Step Exploitation (Path Traversal) . . . . .    | 9        |
| 6.2.5    | HTTP Request Example . . . . .                          | 9        |
| 6.2.6    | Attack Diagram . . . . .                                | 10       |
| 6.2.7    | Impact . . . . .                                        | 10       |
| 6.2.8    | Detection Methodology . . . . .                         | 10       |
| 6.2.9    | Burp Suite Methodology . . . . .                        | 10       |
| 6.2.10   | Prevention . . . . .                                    | 10       |
| 6.2.11   | Pentesting Tips . . . . .                               | 10       |
| 6.3      | Missing <code>state</code> Parameter (CSRF) . . . . .   | 11       |
| 6.3.1    | Theory . . . . .                                        | 11       |
| 6.3.2    | Why It Happens . . . . .                                | 11       |
| 6.3.3    | Attack Prerequisites . . . . .                          | 11       |
| 6.3.4    | Step-by-Step Exploitation (Account Linking) . . . . .   | 11       |
| 6.3.5    | HTTP Request Example . . . . .                          | 11       |
| 6.3.6    | Impact . . . . .                                        | 11       |
| 6.3.7    | Detection Methodology . . . . .                         | 11       |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 6.3.8    | Burp Suite Methodology                 | 11        |
| 6.3.9    | Prevention                             | 12        |
| 6.3.10   | Pentesting Tips                        | 12        |
| 6.3.11   | Important Notes                        | 12        |
| 6.4      | Flawed Scope Validation                | 12        |
| 6.4.1    | Theory                                 | 12        |
| 6.4.2    | Why It Happens                         | 12        |
| 6.4.3    | Attack Prerequisites                   | 12        |
| 6.4.4    | Step-by-Step Exploitation              | 12        |
| 6.4.5    | HTTP Request Example                   | 12        |
| 6.4.6    | Impact                                 | 13        |
| 6.4.7    | Detection Methodology                  | 13        |
| 6.4.8    | Burp Suite Methodology                 | 13        |
| 6.4.9    | Prevention                             | 13        |
| 6.4.10   | Pentesting Tips                        | 13        |
| 6.5      | Unverified User Registration           | 13        |
| 6.5.1    | Theory                                 | 13        |
| 6.5.2    | Why It Happens                         | 13        |
| 6.5.3    | Step-by-Step Exploitation              | 14        |
| 6.5.4    | Attack Scenarios                       | 14        |
| 6.5.5    | Impact                                 | 15        |
| 6.5.6    | Detection Methodology                  | 15        |
| 6.5.7    | Prevention                             | 16        |
| 6.5.8    | Pentesting Tips                        | 16        |
| 6.6      | Access Token Leakage via Open Redirect | 16        |
| 6.6.1    | Theory                                 | 16        |
| 6.6.2    | Why It Happens                         | 16        |
| 6.6.3    | Attack Prerequisites                   | 17        |
| 6.6.4    | Step-by-Step Exploitation (Token Leak) | 17        |
| 6.6.5    | HTTP Request Example                   | 17        |
| 6.6.6    | Impact                                 | 17        |
| 6.6.7    | Detection Methodology                  | 17        |
| 6.6.8    | Prevention                             | 17        |
| 6.7      | Unsafe <code>postMessage</code> Usage  | 17        |
| 6.7.1    | Theory                                 | 17        |
| 6.7.2    | Why It Happens                         | 17        |
| 6.7.3    | Attack Prerequisites                   | 17        |
| 6.7.4    | Step-by-Step Exploitation              | 18        |
| 6.7.5    | HTTP Request Example                   | 18        |
| 6.7.6    | Attack Diagram                         | 18        |
| 6.7.7    | Impact                                 | 18        |
| 6.7.8    | Detection Methodology                  | 18        |
| 6.7.9    | Prevention                             | 18        |
| <b>7</b> | <b>OAuth Pentesting Methodology</b>    | <b>18</b> |
| 7.1      | Reconnaissance                         | 18        |
| 7.2      | Mapping the Flow                       | 18        |
| 7.3      | Testing Redirect URI Validation        | 19        |
| 7.4      | Testing State Parameter                | 19        |
| 7.5      | Testing Scope Validation               | 19        |

---

|     |                                   |    |
|-----|-----------------------------------|----|
| 7.6 | Testing Token Leakage . . . . .   | 19 |
| 7.7 | Testing Account Linking . . . . . | 19 |
| 8   | Prevention and Mitigation         | 19 |
| 9   | Checklist for Penetration Testers | 20 |

# 1 Introduction

OAuth 2.0 is an authorization framework that enables third-party applications to obtain limited access to a user's resources without exposing their credentials. While the protocol itself is robust, real-world implementations often introduce vulnerabilities that attackers can exploit. This document provides a comprehensive guide for penetration testers to understand OAuth internals, identify weaknesses, and perform offensive assessments.

## 1.1 Why OAuth Matters for Pentesters

- OAuth is the backbone of modern authentication (SSO, social login, API protection).
- Implementation flaws are common and often critical, leading to account takeover.
- OAuth tokens and codes are valuable targets; attackers aim to steal them.
- The attack surface is broad: redirect URIs, state parameters, scope validation, client secrets, etc.

This guide covers:

- OAuth fundamentals and components.
- Detailed explanation of flows (Authorization Code, Implicit, etc.).
- Common vulnerabilities with step-by-step exploitation.
- Detection and prevention methodologies.
- Real-world examples and PortSwigger labs walkthroughs.

## 2 OAuth Fundamentals

### 2.1 Core Concepts

OAuth decouples authentication from authorization. The user (Resource Owner) authenticates with the Authorization Server (IdP) and grants permission to the Client Application to access resources on the Resource Server.

### 2.2 OAuth Components

- **Resource Owner** – The user who owns the data and consents to access.
- **Client Application** – The application requesting access (e.g., a web or mobile app).
- **Authorization Server** – Issues tokens after authenticating the user and obtaining consent.
- **Resource Server** – Hosts the protected resources (APIs) that the client wants to access.
- **Identity Provider (IdP)** – Often integrated with the Authorization Server; authenticates the user.
- **User Agent** – The browser or native application used by the resource owner.

## 2.3 OAuth Endpoints

| Endpoint       | Purpose                   | Method | Key Parameters                                       |
|----------------|---------------------------|--------|------------------------------------------------------|
| /authorize     | Initiates the OAuth flow  | GET    | client_id, redirect_uri, response_type, scope, state |
| /token         | Exchanges code for tokens | POST   | grant_type, code, client_id, client_secret           |
| /userinfo      | Returns user data         | GET    | Authorization: Bearer <token>                        |
| /introspection | Validates token status    | POST   | token                                                |
| /revoke        | Revokes a token           | POST   | token                                                |

Table 1: OAuth Endpoints

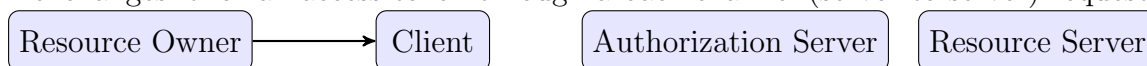
## 2.4 OAuth Tokens

- **Access Token** – Used to access resources. Short-lived (minutes to hours). Stored client-side. Validated by Resource Server.
- **Refresh Token** – Used to obtain new access tokens. Long-lived. Stored by the client (secured). Validated by Authorization Server.
- **Authorization Code** – Temporary code exchanged for tokens. Single-use, short-lived. Sent to client via redirect.
- **ID Token** – JSON Web Token (JWT) containing user identity claims. Used in OpenID Connect.
- **Bearer Token** – Any token that can be used as a bearer credential (e.g., in `Authorization` header).

## 3 OAuth Flows

### 3.1 Authorization Code Grant

**Overview** The most secure flow. The client obtains an authorization code via user consent, then exchanges it for an access token through a back-channel (server-to-server) request.



#### Steps:

1. Client redirects user to Authorization Server with `response_type=code`.
2. User authenticates and consents.
3. Authorization Server redirects back with an authorization code.
4. Client exchanges code for access token (using client secret).
5. Authorization Server returns access token (and optionally refresh token).
6. Client uses access token to access Resource Server.

## 3.2 Implicit Grant

**Overview** Used for public clients (e.g., SPA). The access token is returned directly in the redirect URI fragment. Less secure because token is exposed in the browser.

## 3.3 Other Flows

- **Resource Owner Password Credentials** – Directly sends user credentials to client (not recommended).
- **Client Credentials** – Machine-to-machine communication using client secret only.

## 4 Common Misconceptions

- OAuth is not authentication; it's authorization. OpenID Connect adds authentication.
- The client secret is not needed for public clients (SPAs, mobile apps) because it can't be kept secret.
- The `state` parameter is not optional; it protects against CSRF.
- The `redirect_uri` must be strictly validated; domain-only validation is insufficient.
- Implicit flow is deprecated for new applications; use Authorization Code with PKCE.

## 5 Attack Surface

The OAuth attack surface includes:

- **Redirect URI** – Manipulation leads to code/token leakage.
- **State Parameter** – Missing or predictable leads to CSRF.
- **Scope Parameter** – Flawed validation allows privilege escalation.
- **Client Secret** – Exposure (e.g., in source code) leads to token theft.
- **Authorization Code** – Leakage via referer headers, open redirects, or `postMessage`.
- **Access Token** – Leakage via browser history, logs, or JavaScript.
- **Account Linking** – Flaws allow attackers to associate malicious accounts.
- **User Registration** – Unverified email allows account takeover.

## 6 OAuth Vulnerabilities

### 6.1 Authentication Bypass via OAuth Implicit Flow

#### 6.1.1 Theory

In the implicit flow, the access token is returned in the URL fragment. If the client application does not properly validate the token's origin or audience, an attacker can trick the client into using a token issued for a different application or user.

### 6.1.2 Why It Happens

The client application trusts the token without verifying its intended audience (`aud` claim) or signature, or it accepts tokens from any authorization server.

### 6.1.3 Attack Prerequisites

- Victim uses an OAuth client that accepts tokens from a configured issuer.
- Attacker can obtain an access token for their own account from the same or a different OAuth provider.
- The client does not validate the token's audience or issuer properly.

### 6.1.4 Step-by-Step Exploitation

1. Attacker registers an application with the OAuth provider and obtains an access token for their own account.
2. Attacker intercepts the redirect URI of the victim client and replaces the token with their own.
3. The victim client accepts the token and logs the attacker in as the victim.

### 6.1.5 HTTP Request Example

```
GET /callback#access_token=attacker_token&token_type=Bearer
```

### 6.1.6 Real-World Scenario

A single-page application using implicit flow accepts any token from a trusted provider without checking the `aud` claim. An attacker obtains a token for a different app and uses it to log in.

### 6.1.7 Impact

Account takeover, privilege escalation.

### 6.1.8 Detection Methodology

- Check if the client validates the token's `iss` and `aud` claims.
- Attempt to use a token issued for a different client ID.
- Use Burp to intercept and replace tokens in redirects.

### 6.1.9 Important Notes

- Implicit flow is deprecated; many modern apps use Authorization Code with PKCE, which mitigates this.



## 6.2 Improper redirect\_uri Validation

### 6.2.1 Theory

The `redirect_uri` parameter determines where the authorization code/token is sent. If the Authorization Server does not strictly validate the URI (e.g., only checks the domain), attackers can redirect the code to a malicious endpoint.

### 6.2.2 Why It Happens

Developers often use loose validation (e.g., domain match or prefix match) to allow flexibility, but this opens the door to open redirect and code leakage.

### 6.2.3 Attack Prerequisites

- The Authorization Server does not enforce exact URI matching.
- The client application has registered a URI that can be manipulated via path traversal or subdomain tricks.

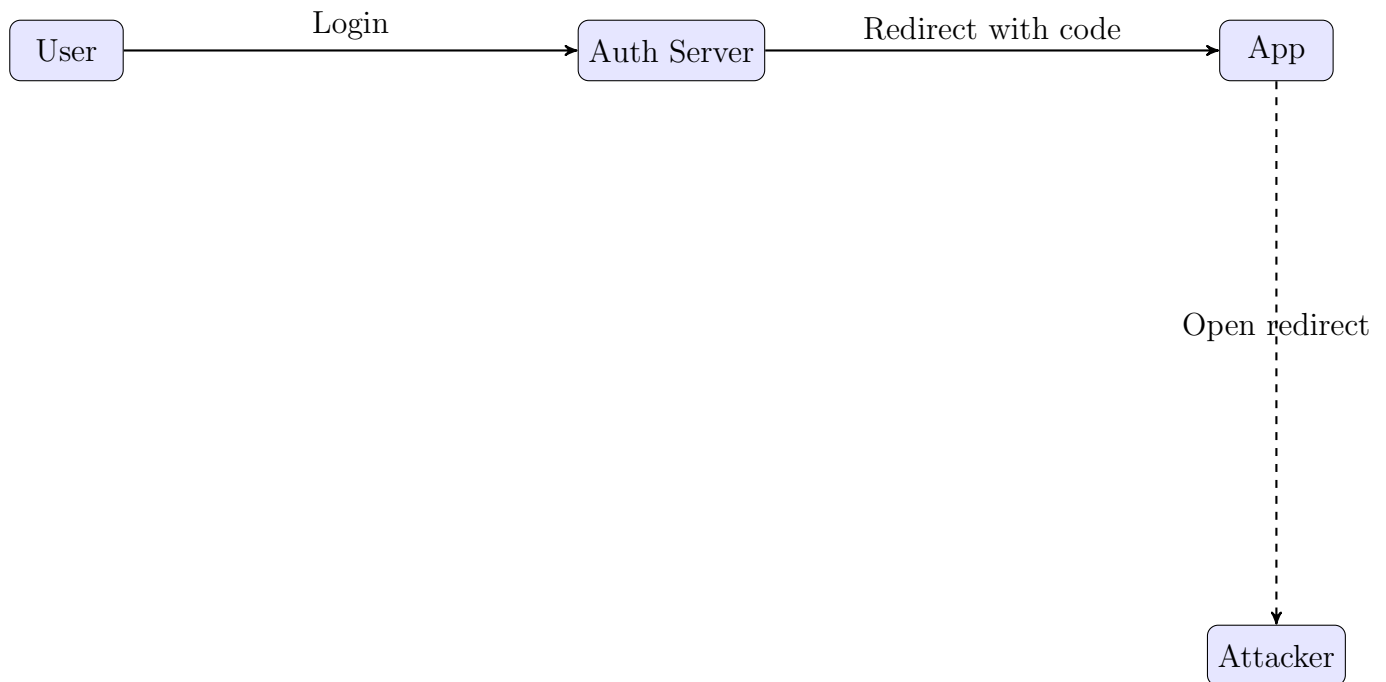
### 6.2.4 Step-by-Step Exploitation (Path Traversal)

1. Find a whitelisted domain, e.g., `https://app.com/oauth-callback`.
2. Construct a malicious `redirect_uri` that uses path traversal to redirect to an open redirect endpoint:  
`https://app.com/oauth-callback/../../post/next?path=https://attacker.com`.
3. If the server allows that, the authorization code will be sent to `https://attacker.com` via the open redirect.
4. Attacker logs the code and uses it.

### 6.2.5 HTTP Request Example

```
GET /auth?client_id=123&redirect_uri=https://app.com/oauth-callback/../../post/next?path=https://attacker.com&response_type=code
```

### 6.2.6 Attack Diagram



### 6.2.7 Impact

Authorization code leakage, account takeover.

### 6.2.8 Detection Methodology

- Try to modify `redirect_uri` to a domain you control.
- Test for path traversal, open redirect chaining, and subdomain takeover.
- Observe if the Authorization Server redirects with a code.

### 6.2.9 Burp Suite Methodology

1. Intercept the `/authorize` request.
2. Change `redirect_uri` to various malicious values.
3. Send and follow redirects to see if code is leaked.

### 6.2.10 Prevention

- Use exact URI matching (including path and query).
- Maintain a whitelist of full redirect URIs.
- Normalize URIs before comparison.

### 6.2.11 Pentesting Tips

- Use Burp's *Intruder* to fuzz the `redirect_uri` with common bypass strings.
- Look for open redirect parameters on the client application to chain.

## 6.3 Missing state Parameter (CSRF)

### 6.3.1 Theory

The `state` parameter is used to prevent CSRF attacks by binding the authorization request to the user's session. If missing or predictable, an attacker can initiate a flow and force a victim to complete it, linking the attacker's account to the victim's.

### 6.3.2 Why It Happens

Developers overlook the `state` parameter or consider it optional.

### 6.3.3 Attack Prerequisites

- Authorization Server does not require `state`.
- The client application does not verify it on callback.

### 6.3.4 Step-by-Step Exploitation (Account Linking)

1. Attacker initiates OAuth flow with their own account, copying the authorization URL.
2. Attacker modifies the `redirect_uri` (if possible) or uses an iframe to force the victim to complete the flow.
3. Victim is redirected and authenticates; the authorization code is sent to the attacker's `redirect_uri` (if not validated) or the attacker's session is used.
4. If the code is sent to the attacker-controlled endpoint, they can exchange it and link the victim's account to their social profile.

### 6.3.5 HTTP Request Example

```
GET /auth?client_id=123&redirect_uri=https://attacker.com&
    response_type=code
```

### 6.3.6 Impact

Account takeover, unauthorized linking of accounts.

### 6.3.7 Detection Methodology

- Check the `/authorize` request for a `state` parameter.
- If present, test if it's predictable (e.g., fixed value, session ID).
- Attempt to initiate the flow without `state` and see if it succeeds.

### 6.3.8 Burp Suite Methodology

1. Intercept the authorization request.
2. Remove or modify the `state` parameter.
3. Observe if the callback still works and if the session is hijacked.

### 6.3.9 Prevention

- Always generate a cryptographically random `state` and validate it on callback.
- Bind `state` to the user's session.

### 6.3.10 Pentesting Tips

- Try to brute-force predictable `state` values.
- Use Burp's CSRF token analysis features.

### 6.3.11 Important Notes

- Even if `state` is present, check if it's validated correctly.

## 6.4 Flawed Scope Validation

### 6.4.1 Theory

The scope parameter defines the permissions requested. If the Authorization Server does not validate the scope during the token exchange against the originally approved scope, an attacker can upgrade the token's permissions.

### 6.4.2 Why It Happens

The token endpoint may ignore the scope from the authorization step and accept a new scope in the token request.

### 6.4.3 Attack Prerequisites

- The attacker controls a client application (can modify token exchange request).
- The Authorization Server does not enforce scope consistency.

### 6.4.4 Step-by-Step Exploitation

1. Client requests authorization with a limited scope (e.g., `openid email`).
2. User approves.
3. Client exchanges authorization code for token, adding a broader scope (e.g., `openid email profile`) in the POST to `/token`.
4. If the server doesn't validate against the original approved scope, it returns a token with the broader scope.

### 6.4.5 HTTP Request Example

```
POST /token
Host: auth-server.com
Content-Type: application/x-www-form-urlencoded

client_id=123&client_secret=secret&grant_type=authorization_code&
code=abc&redirect_uri=https://app.com&scope=openid%20email%20
profile
```

### 6.4.6 Impact

Privilege escalation, unauthorized data access.

### 6.4.7 Detection Methodology

- Capture the token exchange request and modify the `scope` parameter.
- Observe if the returned token grants the expanded scope.

### 6.4.8 Burp Suite Methodology

1. Intercept the `/token` request.
2. Add or modify the `scope` parameter.
3. Forward and test the new token's permissions.

### 6.4.9 Prevention

- Always validate that the requested scope does not exceed the originally approved scope.
- Store the approved scope with the authorization code and enforce it.

### 6.4.10 Pentesting Tips

- Test all scopes that the client might support.

## 6.5 Unverified User Registration

### 6.5.1 Theory

If the OAuth provider allows registration without email verification, an attacker can create an account with the victim's email address. When the victim later tries to log in using that provider, the client may associate the attacker's account with the victim.

### 6.5.2 Why It Happens

The root cause is insufficient identity verification during the registration process:

- **No email verification** - The OAuth provider doesn't send a confirmation email or require clicking a verification link
- **Trusting unverified data** - Client applications blindly trust the email claim from the OAuth provider
- **Using email as unique identifier** - Applications use email as the primary key instead of the OAuth provider's unique `sub` (subject) identifier

### 6.5.3 Step-by-Step Exploitation

#### Phase 1: Pre-Registration (Attacker Prepares)

1. Attacker identifies victim's email address (e.g., `victim@company.com`)
2. Attacker goes to OAuth provider's signup page
3. Attacker registers using victim's email without verification
4. Provider creates the account since email verification is not required
5. Attacker now "owns" that email address in the OAuth provider's system

#### Phase 2: The Attack

1. Victim visits a third-party application
2. Victim clicks "Login with [OAuth Provider]"
3. OAuth flow starts - Victim is redirected to the OAuth provider
4. Victim tries to log in (if they have a real account, they might be blocked)
5. Client application receives the user's data from the OAuth provider
6. Application checks the email - it sees `victim@company.com`
7. Application looks up the user in its database using that email
8. If the attacker already linked this email to an account in the app, the victim might be logged into the attacker's account

### 6.5.4 Attack Scenarios

#### Scenario 1: Pre-emptive Account Takeover

```
Timeline:
Day 1: Attacker registers with victim@company.com on OAuth provider
      (no email verification required)
      -->
Day 1: Attacker uses that OAuth account to log into target app
      Target app creates account with victim@company.com
      -->
Day 2: Victim tries to log in using same OAuth provider
      Target app finds existing account (created by attacker)
      Victim is now in the attacker's account!
```

#### Scenario 2: Denial of Service

```
Day 1: Attacker registers with victim@company.com
      -->
Day 2: Victim tries to register with same OAuth provider
      Provider says: "Email already in use"
      Victim cannot use this OAuth provider at all
```

#### Scenario 3: Account Linking Confusion

```
Day 1: Victim already has an account on target app (password-based)
-->
Day 2: Attacker registers with victim@company.com on OAuth provider
-->
Day 3: Victim tries to "Link Social Account" feature
Victim logs in with OAuth provider
Target app sees email matches existing account
But OAuth account belongs to attacker (same email)
Now attacker's OAuth account is linked to victim's app
account!
-->
Day 4: Attacker logs in using their OAuth account
Since it's linked to victim's app account, attacker is now
logged in as victim!
```

### 6.5.5 Impact

| Impact Level | Description                                |
|--------------|--------------------------------------------|
| Critical     | Complete account takeover                  |
| High         | Ability to impersonate victim              |
| Medium       | Denial of service (victim can't use OAuth) |

### 6.5.6 Detection Methodology

#### Manual Testing Steps:

##### 1. Check OAuth provider registration

- Try to register with a disposable email
- See if you receive a verification email
- Try to log in without verifying

##### 2. Test account takeover

- Register with victim's email (if you control it or it's public)
- Try to use that account on target application

##### 3. Check email\_verified claim

- Intercept OAuth response
- Look for email\_verified field in the userinfo response

##### 4. Test target application's behavior

- Does it check email\_verified claim?
- Does it use sub or email as the unique identifier?

### 6.5.7 Prevention

#### For OAuth Providers:

- Always verify email addresses before allowing account creation
- Send verification email with a token/link
- Mark accounts as unverified until verification is complete
- Don't allow login with unverified email
- Support `email_verified` claim in userinfo endpoint

#### For Client Applications:

- Use `sub` (subject) as unique identifier, NOT email
- Check `email_verified` claim if available
- Require additional verification for sensitive actions
- Implement step-up authentication for high-risk users
- Don't automatically link accounts based on email alone

### 6.5.8 Pentesting Tips

#### 1. Test registration without verification

```
Register -> Check email -> No email received -> Login works?
```

#### 2. Test email takeover

```
Register with victim@domain.com -> Use that account -> Victim  
can't register
```

#### 3. Check OAuth userinfo response

```
GET /userinfo -> Look for email_verified -> If missing or  
false, test further
```

#### 4. Test account linking

```
Try to link OAuth account to an existing account -> What  
happens?
```

## 6.6 Access Token Leakage via Open Redirect

### 6.6.1 Theory

If the `redirect_uri` can point to an open redirect endpoint on the client application, an attacker can steal the token/code by forcing the redirect to their own domain.

### 6.6.2 Why It Happens

The client application has an open redirect vulnerability that is chained with OAuth.



### 6.6.3 Attack Prerequisites

- Client application has an open redirect (e.g., `/next?path=...`).
- OAuth service allows `redirect_uri` to point to that open redirect.

### 6.6.4 Step-by-Step Exploitation (Token Leak)

1. Identify open redirect endpoint on the client:  
`https://app.com/redirect?url=https://attacker.com`.
2. Craft OAuth `redirect_uri` to point to that endpoint:  
`https://app.com/redirect?url=https://attacker.com`.
3. When user completes authorization, the code/token is sent to the open redirect, which forwards it to the attacker's server.

### 6.6.5 HTTP Request Example

```
GET /auth?client_id=123&redirect_uri=https://app.com/redirect?url=
https://attacker.com&response_type=token
```

### 6.6.6 Impact

Token theft, account takeover.

### 6.6.7 Detection Methodology

- Fuzz the `redirect_uri` with different open redirect payloads.
- Check if the Authorization Server follows the redirect and leaks parameters.

### 6.6.8 Prevention

- Strictly validate `redirect_uri` against a whitelist of exact URIs.
- Remove any open redirect vulnerabilities from the client application.

## 6.7 Unsafe `postMessage` Usage

### 6.7.1 Theory

If a client application uses `postMessage` with a wildcard origin and leaks the token in the message, an attacker can intercept it.

### 6.7.2 Why It Happens

The client uses `postMessage` to communicate between frames and sets `targetOrigin` to `*`.

### 6.7.3 Attack Prerequisites

- The application sends the token or full URL (with token) via `postMessage` to `*`.
- Attacker can embed an `iframe` and listen for messages.

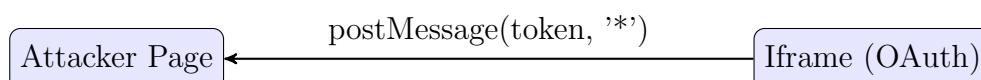
### 6.7.4 Step-by-Step Exploitation

1. Attacker embeds the OAuth login iframe on their page.
2. The iframe loads the login page, which after authentication redirects to a page that sends a `postMessage` containing the token.
3. Attacker's page has an event listener that captures the message and exfiltrates the token.

### 6.7.5 HTTP Request Example

Not applicable.

### 6.7.6 Attack Diagram



### 6.7.7 Impact

Token theft.

### 6.7.8 Detection Methodology

- Check the JavaScript code for `postMessage` calls.
- Look for wildcard origins.

### 6.7.9 Prevention

- Always specify the exact target origin in `postMessage`.
- Validate the origin of incoming messages.

## 7 OAuth Pentesting Methodology

### 7.1 Reconnaissance

- Identify OAuth endpoints:  
`/.well-known/oauth-authorization-server`, `/.well-known/openid-configuration`.
- Find client IDs and redirect URIs from source code or network traffic.
- Determine supported grant types.

### 7.2 Mapping the Flow

- Step through the OAuth flow with Burp proxy.
- Note parameters: `client_id`, `redirect_uri`, `response_type`, `scope`, `state`.
- Identify all endpoints involved.

### 7.3 Testing Redirect URI Validation

- Modify `redirect_uri` to different domains.
- Use path traversal, subdomain variants, open redirects.
- Observe if code/token is leaked.

### 7.4 Testing State Parameter

- Remove `state` and see if flow still works.
- Predict `state` if generated from session IDs or timestamps.
- Attempt CSRF by using a crafted URL.

### 7.5 Testing Scope Validation

- Capture token exchange and modify scope.
- Test if token grants expanded permissions.

### 7.6 Testing Token Leakage

- Check referer headers for token.
- Test if token appears in logs, browser history, or `postMessage`.

### 7.7 Testing Account Linking

- Attempt to link a social account to an existing account.
- Check if the linking is CSRF-protected.

## 8 Prevention and Mitigation

- Use Authorization Code flow with PKCE for public clients.
- Strictly validate `redirect_uri` with exact matching.
- Always use a cryptographically random `state` parameter and validate it.
- Validate scope consistency during token exchange.
- Use OpenID Connect for authentication with proper ID token validation.
- Keep client secrets secure; use OAuth 2.0 for server-side apps only.
- Implement token revocation and short-lived tokens.
- Use introspection endpoints to validate tokens.

## 9 Checklist for Penetration Testers

1. Identify all OAuth endpoints and grant types.
2. Enumerate client IDs and registered redirect URIs.
3. Test `redirect_uri` validation:
  - Change domain.
  - Change path.
  - Add extra parameters.
  - Use path traversal.
  - Use open redirect endpoints.
4. Test `state` parameter:
  - Remove it.
  - Use a predictable value.
  - Test CSRF by forcing victim.
5. Test scope validation:
  - Modify scope in token exchange.
  - Verify token permissions.
6. Test token leakage:
  - Check referer header.
  - Check browser history.
  - Check `postMessage` usage.
7. Test account linking:
  - Attempt to link an account with CSRF.
  - Attempt to link without verification.
8. Test for unverified user registration on the OAuth provider.
9. Test client secret exposure (if applicable).
10. Test for authorization server confusion (multiple issuers).